

BAHRIA UNIVERSITY KARACHI CAMPUS

DEPARTMENT OF COMPUTER SCIENCE



Bahria University
Discovering Knowledge

COMPUTER GRAPHICS LAB (CSL-444)

BS IT 6A – Spring 2026

Project Title:

INTERACTIVE SOLAR SYSTEM SIMULATION

Submitted by:

MAHNOOR (02-235232-001)

FAIZA ALTAF (02-235232-036)

HADIQA MEHMOOD (02-235232-007)

Submitted To:

Sir M. Usama

Submitted On:

18th May 2026

1. Acknowledgement

We would like to express our sincere gratitude to our Computer Graphics Lab instructor, Sir M. Usama, for his valuable guidance, encouragement, and continuous support throughout the development of this project.

We are also thankful to the Department of Computer Science, Bahria University, for providing us with the opportunity and resources to work on this project. This project helped us strengthen our understanding of Computer Graphics concepts including OpenGL rendering, transformations, lighting, animation, and user interaction.

Special thanks to all group members, Faiza Altaf, Hadiqa Mehmood, and Mahnoor Sharif, for their teamwork, dedication, and collaborative efforts in completing this project successfully.

2. Abstract

The “Interactive Solar System Simulation” is a 3D Computer Graphics project developed using C++, OpenGL, and FreeGLUT. The purpose of this project is to simulate the solar system in an interactive and visually appealing environment while demonstrating important Computer Graphics concepts.

The simulation includes the Sun, planets, moons, asteroid belts, Kuiper belt, comet effects, starfield backgrounds, lighting systems, and smooth orbital animations. Users can interact with the system through keyboard and mouse controls for camera rotation, zooming, speed adjustment, fullscreen mode, and planet information panels.

Advanced graphics concepts such as hierarchical modeling, lighting and shading, transformations, animation, blending, transparency effects, and camera systems are implemented in the project. The simulation also contains a drone camera system, HUD interface, atmospheric effects, and realistic planetary rendering.

This project demonstrates how Computer Graphics techniques can be applied to create educational and interactive scientific visualizations in a real-time environment.

3. Table of contents

1. Acknowledgement	2
2. Abstract.....	2
3. Table of contents	3
4. Introduction	4
5. Problem statement	4
6. Existing Systems.....	5
7. Objectives and goals.....	5
8. Project Scope	5
9. Workflow	6
10. Project Overview	8
11. Tools and Technologies	8
12. Project features.....	8
13. Computer Graphics implemented concepts	9
14. Output	11
15. Future work.....	14
16. Conclusion	14
17. References.....	14

4. Introduction

Computer Graphics is an important field of computer science that focuses on generating and manipulating visual content using computers. It is widely used in simulations, gaming, education, scientific visualization, and virtual reality systems.

Understanding the solar system through static images and textbook diagrams is difficult because planetary motion, rotation, and spatial relationships cannot be visualized effectively. Therefore, interactive simulations are useful for improving understanding and engagement.

The “Interactive Solar System Simulation” project is designed to create a real-time 3D visualization of the solar system using OpenGL and FreeGLUT libraries in C++. The project models planets, moons, orbital motion, lighting effects, and camera interaction in a dynamic environment.

The project demonstrates both fundamental and advanced Computer Graphics concepts such as:

- 3D transformations
- Hierarchical modeling
- Real-time animation
- Lighting and shading
- Camera systems
- Double buffering
- User interaction

The simulation provides an educational and visually attractive environment where users can explore the solar system interactively.

5. Problem statement

Traditional learning methods use static diagrams and images to explain the solar system, which makes it difficult to understand the movement, scale, and relationships between celestial bodies.

There is a need for an interactive and visually realistic simulation that can:

- Demonstrate planetary rotation and revolution
- Show hierarchical relationships between planets and moons
- Provide smooth animation and real-time interaction
- Improve user understanding through visualization
- Implement important Computer Graphics concepts practically

This project solves these problems by developing a fully interactive 3D solar system simulation using OpenGL.

6. Existing Systems

Table 6.1: Comparison of Existing Systems

Existing System	Features	Limitations
Textbook Diagrams	Simple representation of planets	No animation or interaction
2D Solar Simulations	Basic orbital motion	Lack of realism and 3D environment
NASA Visualizations	Highly realistic graphics	Complex and resource intensive
Unity/Unreal Simulators	Advanced rendering and physics	Requires external game engines
Proposed System	Interactive 3D OpenGL simulation with lighting, animation, HUD, drone camera, and effects	Simplified astronomical accuracy

7. Objectives and goals

Primary Objectives

- To develop a real-time interactive 3D solar system simulation
- To implement Computer Graphics concepts using OpenGL
- To create smooth planetary animation and orbital motion
- To provide interactive camera and user controls
- To create an educational and visually attractive simulation

Goals

- Implement hierarchical modeling for planets and moons
- Apply transformations including translation, rotation, and scaling
- Demonstrate lighting and shading techniques
- Create responsive keyboard and mouse interaction
- Ensure smooth rendering and animation performance

8. Project Scope

In-Scope Features

- Sun as central light source
- Planetary revolution and rotation
- Moon systems
- Orbit rings
- Lighting and shading
- Atmospheric effects
- Asteroid belt and Kuiper belt

- Drone camera mode
- HUD interface
- Interactive controls
- Full screen mode
- Planet information panel
- Starfield background
- Real-time animation

Out-of-Scope Features

- Real-world astronomical calculations
- Gravity physics simulation
- Collision detection
- External game engines
- Advanced AI systems

9. Workflow

The system workflow of the Interactive Solar System Simulation is as follows:

1. Start program and initialize OpenGL environment.
2. Load required libraries (GLUT, rendering settings, camera setup).
3. Initialize scene components:
 - Sun
 - Planets
 - Moons
 - Asteroid belt
 - Starfield background
4. Set lighting system (ambient, diffuse, specular).
5. Initialize camera system (orbit view by default).
6. Enter main rendering loop.
7. Continuously perform:
 - Update planetary rotation
 - Update orbital revolution
 - Update camera movement (if user interacts)
 - Render all objects
8. Capture user input:
 - Keyboard controls (pause, speed, views)
 - Mouse controls (rotation, zoom)
9. Update HUD and information panels if requested.
10. Swap buffers for smooth animation.
11. Repeat loop until exit command is triggered.
12. Terminate program safely.

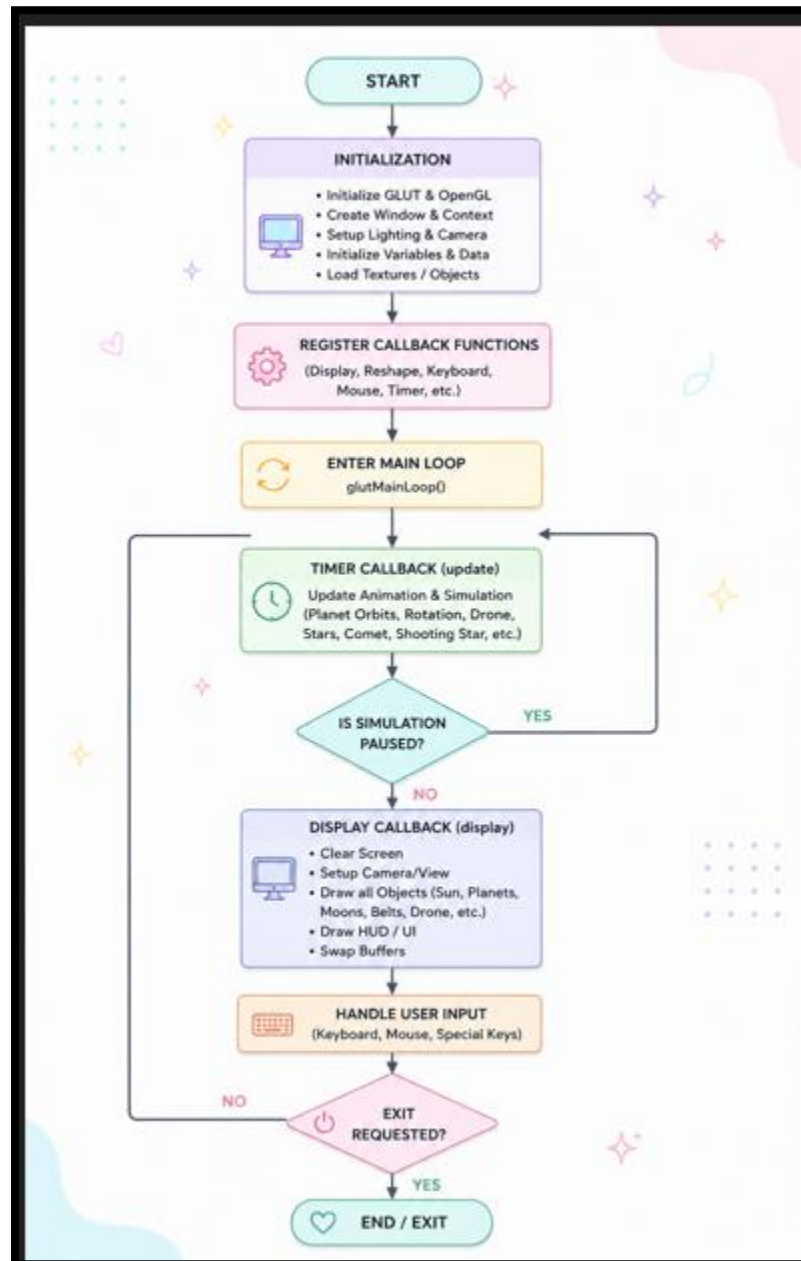


Figure 9.1: Flow Chart

10. Project Overview

The project is an interactive OpenGL simulation that visually represents the solar system in a realistic 3D environment.

The simulation includes:

- Eight planets with unique rendering
- Planetary rotation and revolution
- Moon systems using hierarchical modeling
- Sun lighting effects
- Atmospheric glow effects
- Asteroid belt and Kuiper belt
- Shooting stars and comet effects
- Interactive drone camera
- HUD interface with planet information
- Mouse and keyboard controls

The project uses modular programming where separate functions are responsible for rendering different celestial objects and effects.

The rendering loop continuously updates the scene using timer-based animation for smooth movement and interactivity.

11.Tools and Technologies

Table 11.1: Tools and Technologies used in the system

Tool / Technology	Purpose
C++	Main programming language
OpenGL	Graphics rendering
FreeGLUT	Windowing and input handling
Visual Studio/VS Code	Development environment
GLUT Functions	Interaction and rendering utilities
Matrix Transformations	Object movement and hierarchy
Double Buffering	Smooth rendering

12. Project features

1. **Interactive Solar System:** Real-time simulation of planets and moons
2. **Realistic Animation:** Smooth orbital motion and rotation
3. **Lighting Effects:** Ambient, diffuse, and specular lighting

4. **Atmospheric Glow:** Atmospheric rendering around planets
5. **Drone Camera System:** Dynamic camera movement around the scene
6. **HUD Interface:** Planet information and simulation controls
7. **Asteroid Belt:** Animated asteroid system between planets
8. **Shooting Stars and Comets:** Additional space effects for realism
9. **User Interaction:** Keyboard and mouse controls
10. **Full screen and Top View Modes:** Multiple viewing perspectives

13. Computer Graphics implemented concepts

1. 3D Transformations

The project uses:

- Translation
- Rotation
- Scaling

These transformations are used for orbital motion and object positioning.

2. Hierarchical Modeling

Parent-child relationships are implemented:

- Sun → Planet → Moon

This is achieved using matrix stack operations.

3. Lighting and Shading

The project implements:

- Ambient lighting
- Diffuse lighting
- Specular lighting

The Sun acts as the primary light source.

```
class LightingManager {
private:
    vector<Label3D> labelList;
    double g_mv[16], g_proj[16];
    int g_vp[4];
    float nebulaPhase;
    float planetGlowPhase[9];
    float sunPulse;
    bool showLabels;
    bool showOrbits;

public:
    LightingManager();
    ~LightingManager();
}
```

4. Double Buffering

Double buffering is used to eliminate flickering and provide smooth animation.

```
void mouseMotion(int x, int y)
{
    if (mouseDown) {
        if (droneCamera) {
            droneCamAngleY+=(x-lastMouseX)*0.5f;
            droneCamAngleX+=(y-lastMouseY)*0.5f;
        } else {
            camAngleY+=(x-lastMouseX)*0.5f;
            camAngleX+=(y-lastMouseY)*0.5f;
            if(camAngleX> 88) camAngleX= 88;
            if(camAngleX<-88) camAngleX=-88;
        }
        lastMouseX=x; lastMouseY=y;
        glutPostRedisplay();
    }
}
```

5. Real-Time Animation

Timer functions continuously update:

- Planet rotations
- Orbital revolutions
- Camera movement
- Space effects

6. Camera System

Different camera modes are implemented:

- Orbit camera
- Drone camera
- Top-down view

7. Blending and Transparency

Transparency effects are used for:

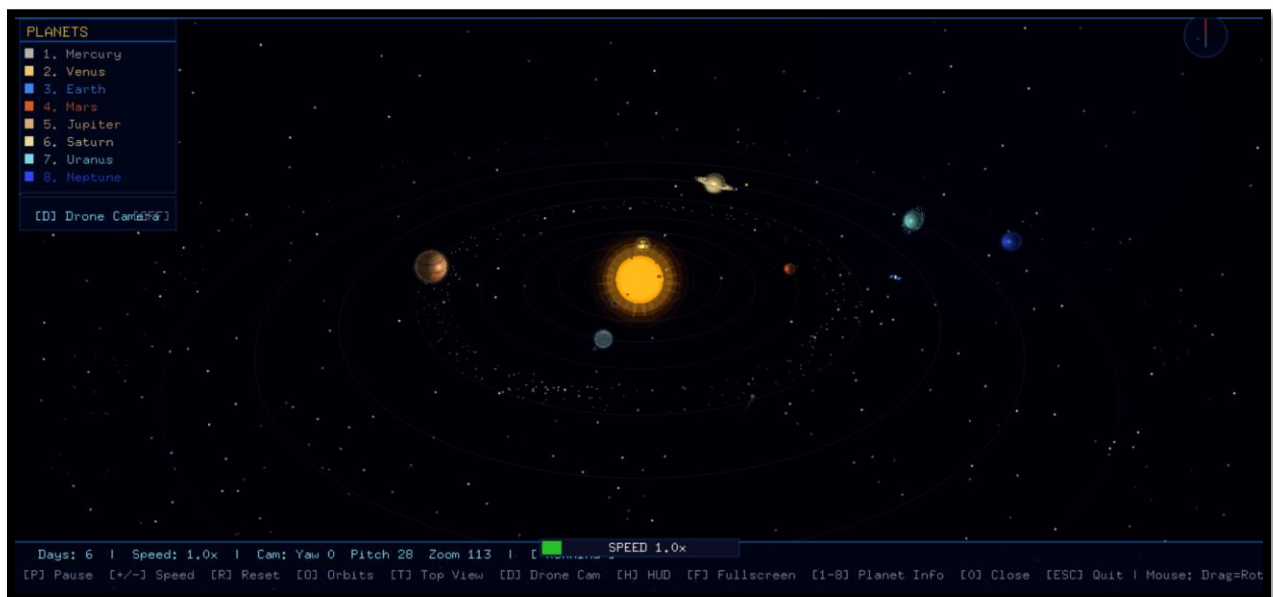
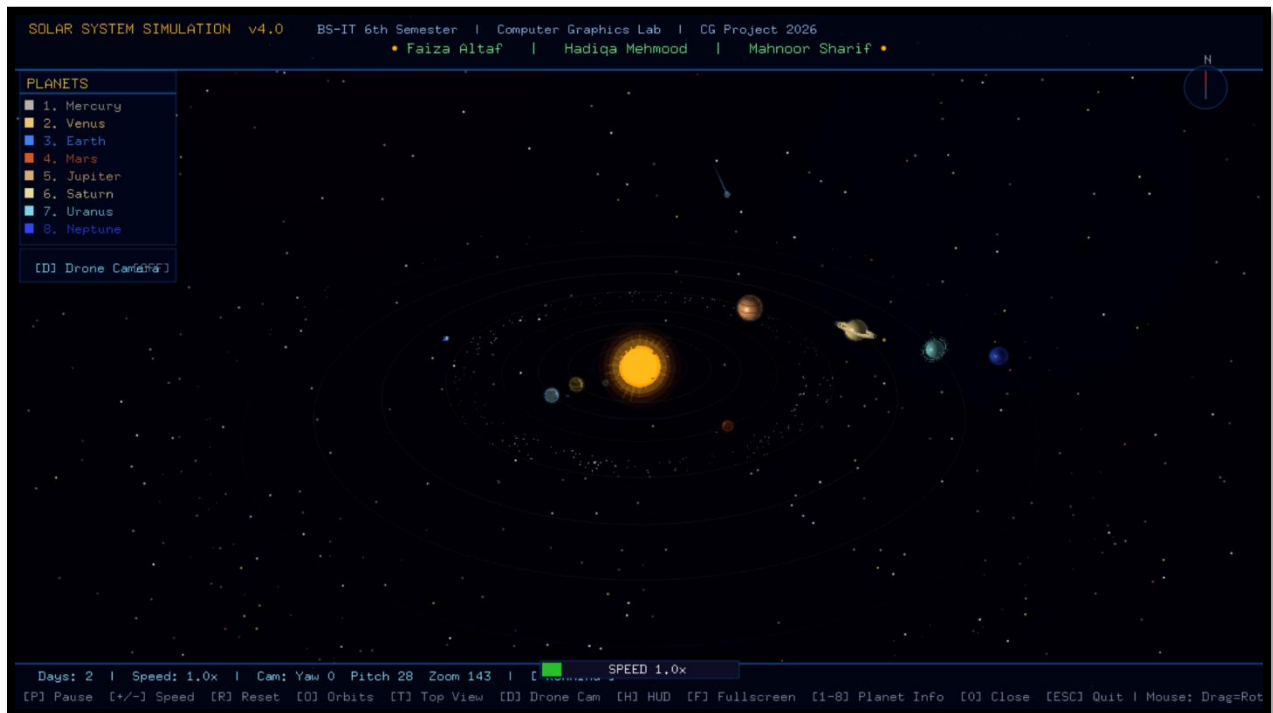
- Planet atmospheres
- Glow effects
- Nebula backgrounds

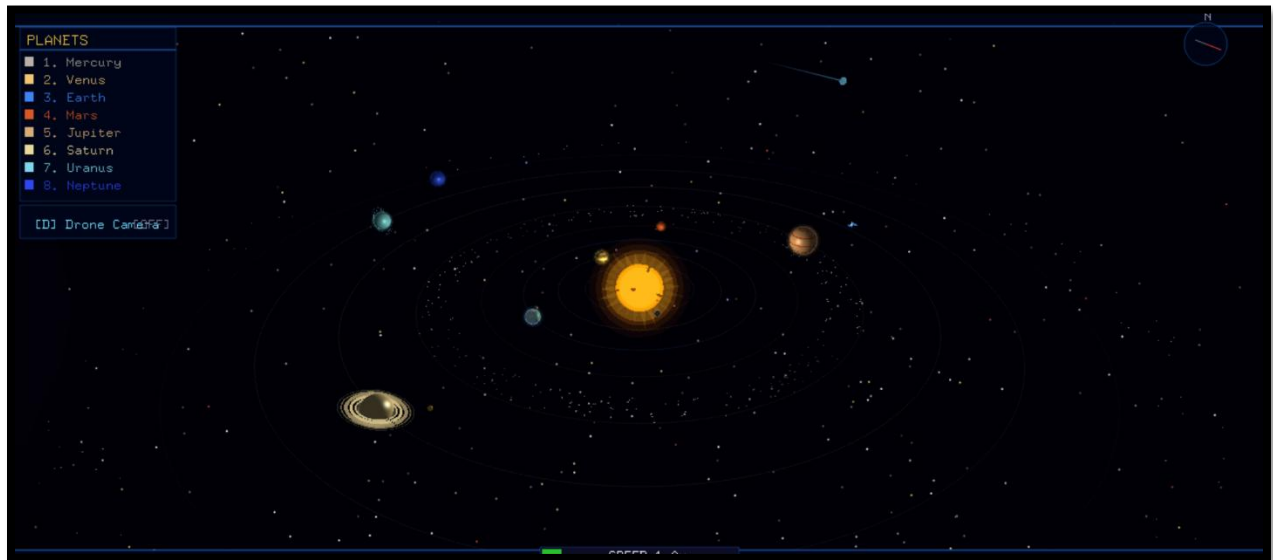
8. Procedural Rendering

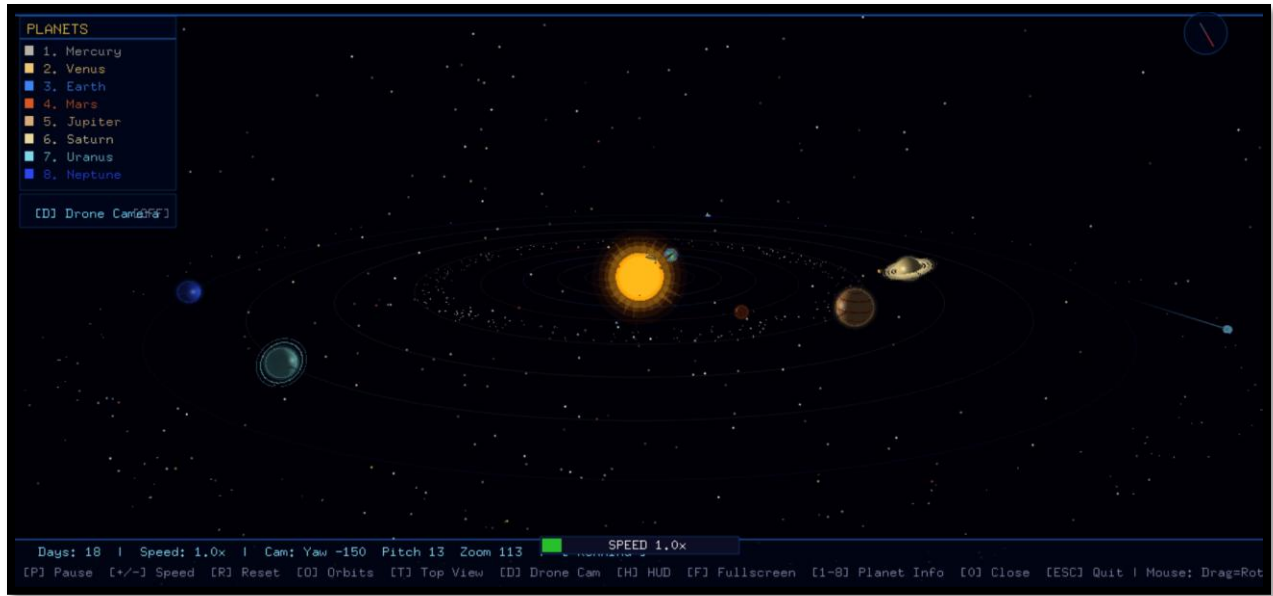
Planet surfaces are generated using mathematical calculations and noise patterns instead of textures.

14. Output

The output of the project is a fully interactive 3D solar system simulation window.







User Controls:

Table 14.1: User Controls

Key	Function
P	Pause/Resume
+ / -	Increase/Decrease speed
O	Toggle orbits
D	Drone camera
T	Top-down view
H	Toggle HUD
F	Fullscreen
1-8	Planet information
0	Close panel
ESC	Exit

Mouse controls are used for:

- Camera rotation
- Zooming
- Viewing angle adjustment

15. Future work

The following enhancements can be added in future versions of the system:

- Add realistic texture mapping using image textures
- Implement accurate astronomical scaling
- Add gravity and physics simulation
- Introduce spacecraft navigation
- Add sound effects and background music
- Implement VR support for immersive experience
- Add galaxy and exoplanet systems
- Improve graphical realism using shaders.

16. Conclusion

The “Interactive Solar System Simulation” project successfully demonstrates the implementation of important Computer Graphics concepts using OpenGL and FreeGLUT in C++.

The project provides a realistic and interactive visualization of the solar system with smooth animation, lighting, hierarchical modeling, camera systems, and user interaction. It also includes advanced enhancements such as asteroid belts, atmospheric effects, drone camera mode, HUD interfaces, and comet animations.

This project improved our understanding of OpenGL programming, graphics rendering, transformations, lighting systems, and real-time animation techniques. The final system is educational, visually attractive, interactive, and fulfills the objectives of the Computer Graphics Lab course.

17. References

- [1] J. Neider, T. Davis, and M. Woo, *OpenGL Programming Guide*, 9th ed. Boston, MA, USA: Addison-Wesley, 2013.
- [2] D. Shreiner, G. Sellers, J. Kessenich, and B. Licea-Kane, *OpenGL SuperBible*, 7th ed. New York, NY, USA: Addison-Wesley, 2015.
- [3] E. Angel and D. Shreiner, *Interactive Computer Graphics: A Top-Down Approach with OpenGL*, 6th ed. Boston, MA, USA: Pearson, 2012.
- [4] F. S. Hill and S. M. Kelley, *Computer Graphics Using OpenGL*, 3rd ed. Upper Saddle River, NJ, USA: Pearson, 2007.
- [5] K. Sung, *Computer Graphics with OpenGL*, Seoul, South Korea: Hanbit Media, 2015.
- [6] The Khronos Group, “OpenGL Official Documentation,” [Online]. Available: [OpenGL Official Website](#)
- [7] [FreeGLUT Official Documentation](#)